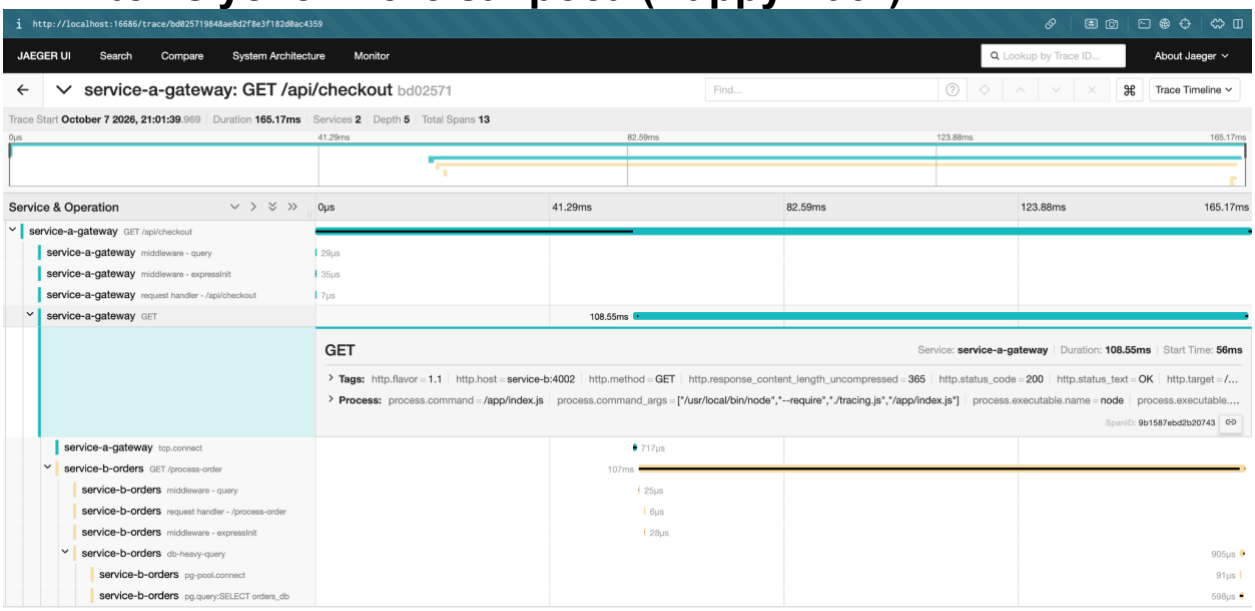
1. Анализ успешного запроса (Happy Path)



1. Количество спанов. Трасса содержит 13 spans, что отображается в верхней части Jaeger UI в поле Total Spans.

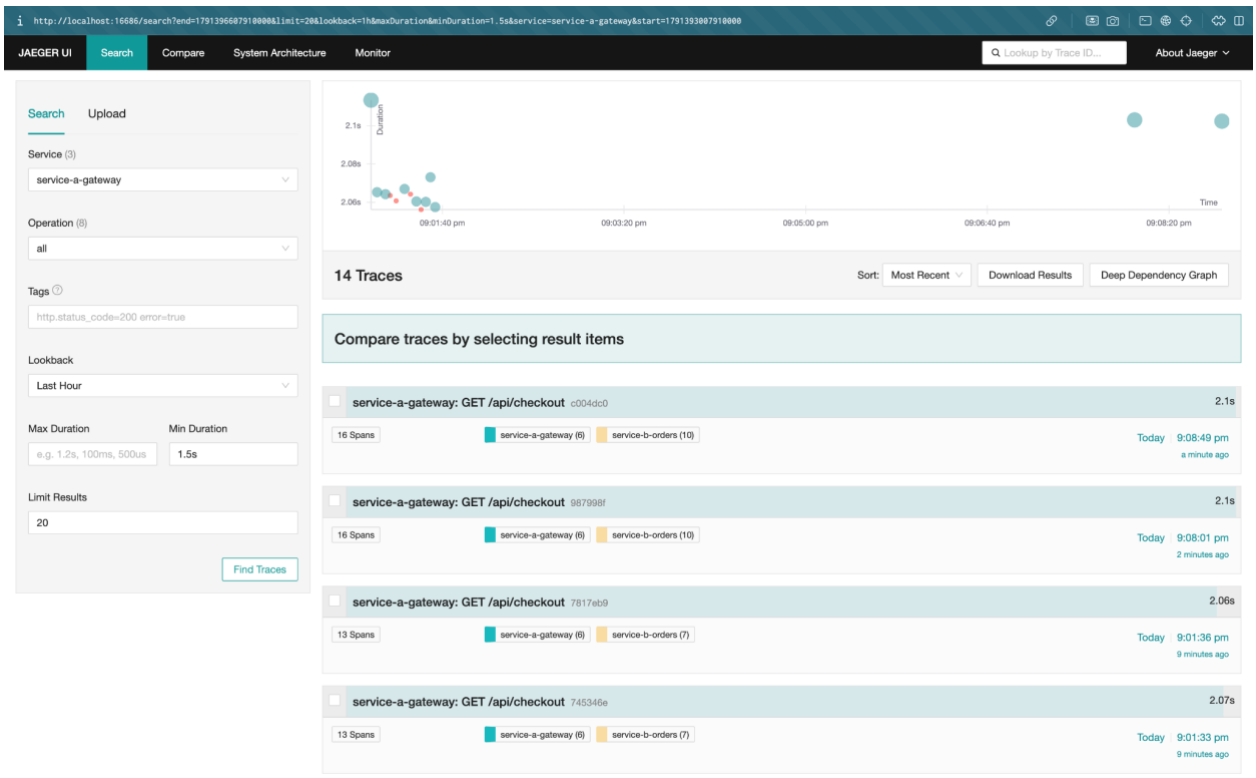
2. Передача контекста трассировки. Контекст между `service-a-gateway` и `service-b-orders` передается HTTP-заголовком `traceparent`. Так как Jaeger UI не показывает его значение напрямую, в обработчик `/process-order` сервиса B временно был добавлен вывод `req.headers['traceparent']` в консоль. После пересборки контейнера и выполнения нового запроса значение было получено через логи Docker:

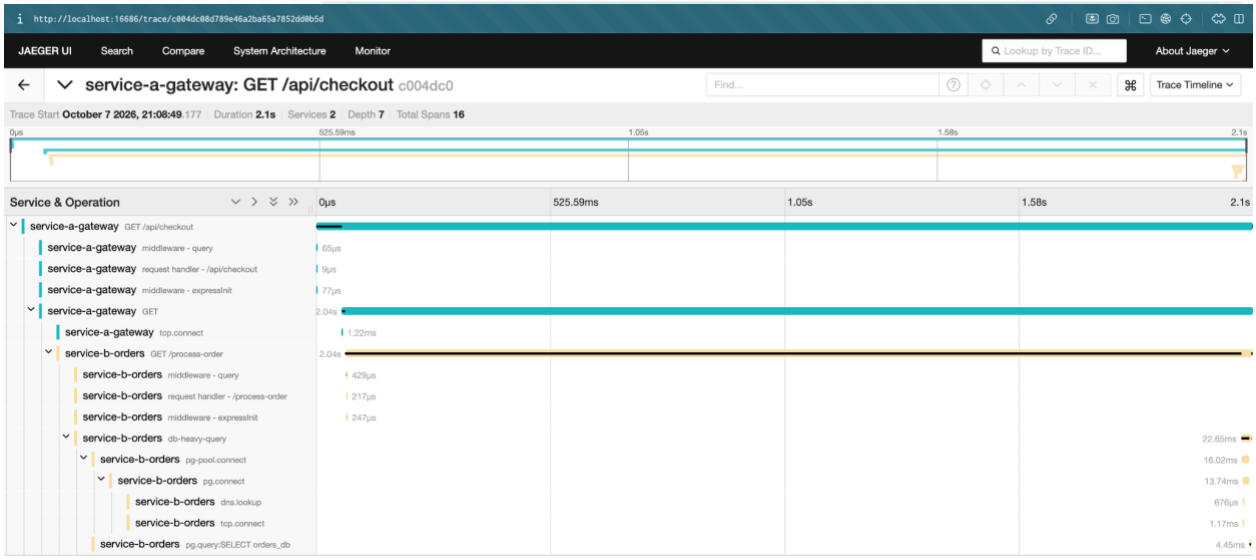
```
traceparent: 00-c004dc08d789e46a2ba65a7852dd0b5d-a92ae8aa4bbda937-01
```

Полученное значение относится к отдельному тестовому запросу, использованному для проверки способа передачи контекста.

3. Время SQL-запроса. Непосредственный спан `pg.query:SELECT orders_db` занял 598 мкс (около 0,598 мс). Следовательно, обращение к PostgreSQL занимает лишь небольшую часть времени успешного запроса.

2. Поиск узкого места (Bottleneck / Latency Analysis)





1. Место возникновения задержки. Узкое место находится в сервисе `service-b-orders`, в обработчике `GET /process-order`. Его спан занимает около 2,04 с. Столько же практически ожидает исходящий HTTP-вызов из service-a-gateway.

2. SQL или код приложения? Причина находится в Node.js-коде, а не в PostgreSQL. На таймлайне видно, что `db-heavy-query` занимает около 22,65 мс, а непосредственно `pg.query:SELECT orders_db` - около 4,45 мс. DB-спаны появляются только в самом конце двухсекундного HTTP-спана.

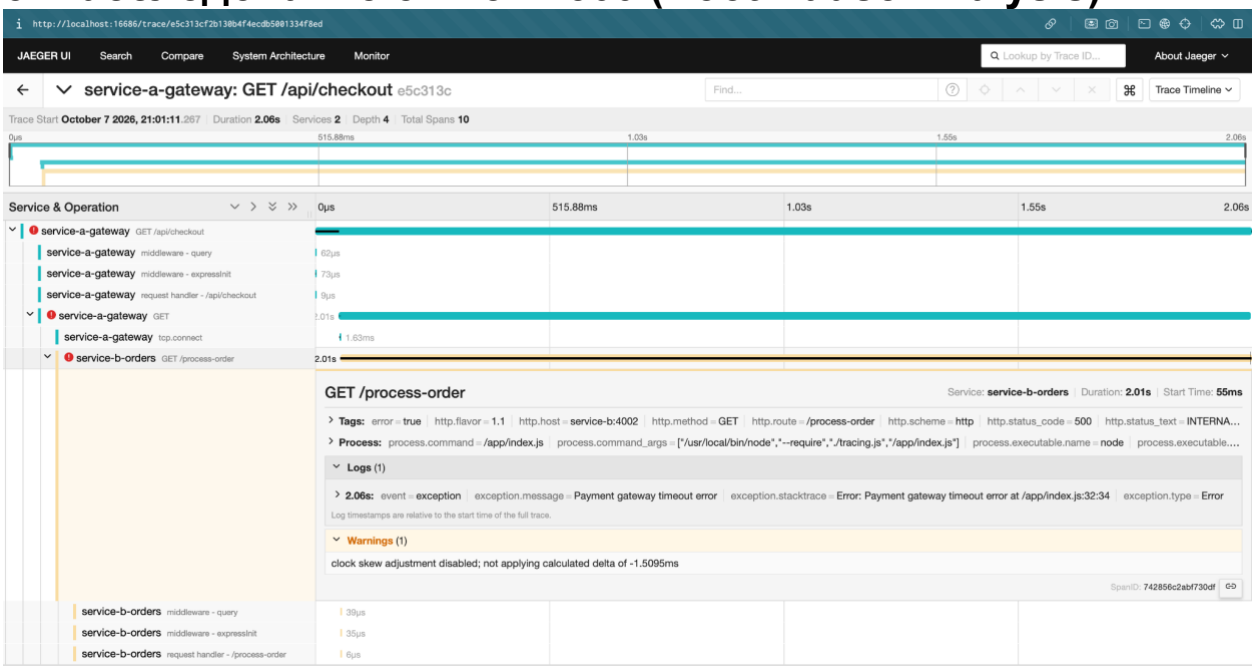
Причиной является искусственная задержка в service-b:

```
const isSlow = Math.random() < 0.4;

const delay = isSlow ? 2000 : 100;

await new Promise((resolve) => setTimeout(resolve, delay));
```

3. Расследование ошибки 500 (Root Cause Analysis)



1. Источник ошибки. Ошибка сгенерирована сервисом `service-b-orders` в обработчике `GET /process-order`. Спан помечен как Error, а HTTP-статус равен 500.

2. Текст исключения. В логах спана записано `exception.message = Payment gateway timeout error`. Также Jaeger показывает тип исключения Error и stack trace, указывающий на `/app/index.js`.

3. Дошел ли запрос до PostgreSQL? Нет. В ошибочной трассе отсутствуют спаны `db-heavy-query`, `pg-pool.connect` и `pg.query`. В коде при срабатывании isError выполняется возврат HTTP 500 до блока с `pool.query()`.

4. Исправление и проверка

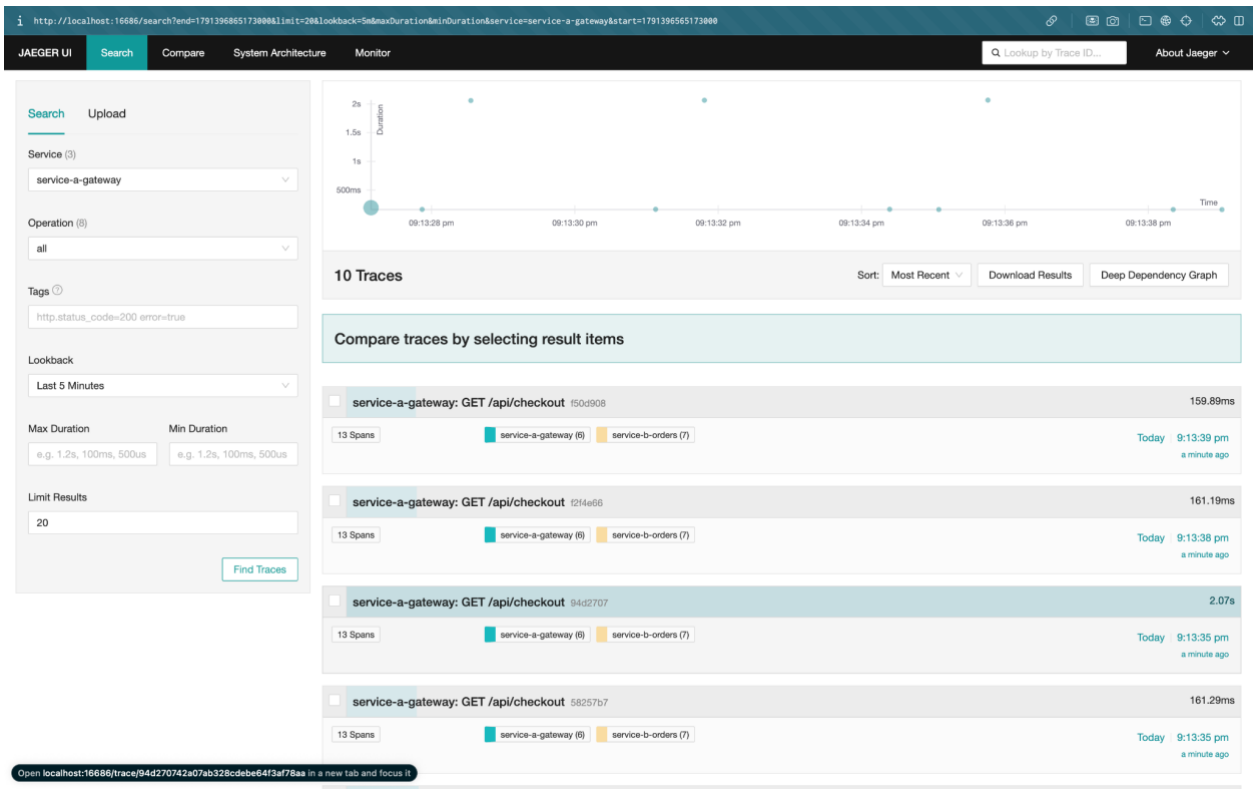
Для устранения искусственной ошибки условие

```
const isError = Math.random() < 0.2;
```

было заменено на

```
const isError = false;
```

После изменения сервис B был пересобран и запущен заново. Затем были выполнены 10 повторных запросов к service-a. Все 10 завершились успешно.



На графике остаются как быстрые трассы около 160 мс, так и медленные около 2,07 с. Это ожидаемо: исправление отключает только генерацию ошибки 500, но не меняет отдельный механизм искусственной задержки `isSlow`.

Вывод

Анализ Waterfall показал, что двухсекундная задержка возникает внутри service-b до обращения к PostgreSQL: сам SQL-запрос выполняется за единицы миллисекунд. Источником ответов 500 также является service-b-orders; в ошибочном спане зафиксировано исключение Payment gateway timeout error, причем запрос завершается до обращения к базе данных.

После отключения искусственного условия ошибки и пересборки service-b все 10 контрольных запросов завершились успешно, а новые трассы перестали содержать красные ошибки. Таким образом, причина сбоя была определена по распределенной трассировке и исправление подтверждено повторным тестированием.